Module 15: Circular Singly Linked Lists

1. Current Progress Analysis

You are ready for Module 15 because you already know the main ideas from normal singly linked lists.

Before this module, you learned:

Previous idea	Why it matters now
Node basics	A node stores `data` and `next`.
Pointer movement	`p = p->next` moves from one node to the next node.
Traversal	A normal singly linked list is visited until `NULL`.
Creation	A list can be built using `first`, `last`, and `t`.
Recursion	A recursive function can move through a list by calling itself with `p->next`.
Insertion and deletion	Linked-list operations mainly work by changing links.
Reversal	Links can be changed carefully without losing nodes.
Loop detection	Not every linked list necessarily ends at `NULL`.
C++ class implementation	Linked-list data and operations can be organized inside a class.

Up to Module 14, most lists looked like this:

```
10 -> 20 -> 30 -> NULL
```

This is a **linear singly linked list**.

Module 15 introduces a different structure:

```
10 -> 20 -> 30
^           |
|___________|
```

This is a **circular singly linked list**.

The biggest change is this:

```
A normal singly linked list stops at NULL.
A circular singly linked list comes back to the first node.
```

---

2. Big Idea of Module 15

A **circular singly linked list** is a singly linked list where the last node points back to the first node.

Normal singly linked list:

```
Head
 |
 v
10 -> 20 -> 30 -> NULL
```

Circular singly linked list:

```
Head
 |
 v
10 -> 20 -> 30
^           |
|___________|
```

The important link is:

```
last->next = Head;
```

In a normal singly linked list, the last node has:

```
last->next = NULL;
```

In a circular singly linked list, the last node has:

```
last->next = Head;
```

That one link creates the circle.

---

3. Linear Singly Linked List vs Circular Singly Linked List

Feature	Linear singly linked list	Circular singly linked list
Last node points to	`NULL`	`Head`
Natural stopping point	`NULL`	Returning to `Head`
Traversal condition	`p != NULL`	`p != Head` after at least one visit
Shape	Straight line	Circle
Risk	Ends normally	Can loop forever if displayed incorrectly

Linear list:

```
10 -> 20 -> 30 -> NULL
```

Circular list:

```
10 -> 20 -> 30
^           |
|___________|
```

---

4. Why We Use the Name `Head`

In earlier modules, the first node pointer was often called:

```c
struct Node *first = NULL;
```

For circular linked lists, it is common to use:

```c
struct Node *Head = NULL;
```

The name `Head` is useful because a circular list does not have a natural end.

So `Head` means:

```
Start traversal here.
Stop when you come back here.
```

In a linear list, `first` means the first node of a line.

In a circular list, `Head` is more like a fixed starting point in a loop.

---

5. Node Structure

A circular singly linked list uses the same kind of node as a normal singly linked list.

C node definition:

```
struct Node {
    int data;
    struct Node *next;
};
```

Each node has two parts:

Part	Meaning
`data`	The value stored in the node
`next`	The address of the next node

Example node:

```
[data | next]
```

The node structure does not change.

What changes is the final link.

Normal list final node:

```
[30 | NULL]
```

Circular list final node:

```
[30 | address of Head]
```

---

6. Standard Circular Singly Linked List Representation

The standard circular singly linked list has three important cases.

Case 1: Empty Circular List

An empty list has no nodes.

```
Head = NULL
```

In code:

```
struct Node *Head = NULL;
```

Meaning:

```
There is no first node.
There is no last node.
The list is empty.
```

Case 2: One-Node Circular List

A one-node circular list looks like this:

```
Head
 |
 v
10
^|
||
|_|
```

The node points to itself.

In code:

```
Head->next = Head;
```

This means:

```
The node after Head is Head itself.
```

Important:

```
A one-node circular list is not empty.
```

Empty list:

```
Head == NULL
```

One-node circular list:

```
Head != NULL
Head->next == Head
```

Case 3: Multiple-Node Circular List

Example:

```
Head
 |
 v
10 -> 20 -> 30
^           |
|___________|
```

The last node points back to `Head`.

If `30` is the last node, then:

```
30->next = Head;
```

Conceptually:

```
After 30, the list goes back to 10.
```

---

7. Dummy-Head Circular List Representation

There is another way to represent a circular linked list. It is called a **dummy-head circular list**.

In this representation, `Head` is not a real data node.

It is a permanent dummy node.

Empty Dummy-Head Circular List

In code:

```
Head->next = Head;
```

This means:

```
The dummy Head node points to itself.
There are no real data nodes.
```

Standard vs Dummy-Head Representation

Representation	Empty list condition	Meaning
Standard circular list	`Head == NULL`	No nodes exist
Dummy-head circular list	`Head->next == Head`	Dummy node exists, but no real data node exists

Main Difference

In the standard version:

```
Head points to the first real node.
```

In the dummy-head version:

```
Head is a special dummy node before the real data nodes.
```

For Module 15, focus mainly on the **standard circular singly linked list**.

---

8. Creating a Circular Singly Linked List

Suppose we want to create this circular linked list:

```
2 -> 3 -> 4 -> back to 2
```

Using this array:

```
int A[] = {2, 3, 4};
```

The final structure should be:

```
Head
 |
 v
2 -> 3 -> 4
^         |
|_________|
```

---

9. Pointers Used During Creation

During creation, we use three main pointers.

Pointer	Meaning
`Head`	Points to the first node of the circular list
`last`	Points to the current last node
`t`	Temporary pointer for the new node being created

The role of `last` is very important.

It helps us attach every new node at the end.

---

10. Creation Logic in Simple Words

To create a circular linked list from an array:

1. Create the first node separately.
2. Make the first node point to itself.
3. Set `last = Head`.
4. For every remaining array value:
5. Create a new node `t`.
6. Put the array value inside `t->data`.
7. Make `t->next = Head`.
8. Attach the old last node to `t` using `last->next = t`.
9. Move `last` to `t`.

The most important circular link is:

```
t->next = Head;
```

This makes each newly added node point back to the first node until another node is added after it.

---

11. C Code: Create Circular Singly Linked List

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *Head = NULL;

void create(int A[], int n) {
    if (n <= 0) {
        Head = NULL;
        return;
    }

    struct Node *t;
    struct Node *last;
```

```
    Head = (struct Node *)malloc(sizeof(struct Node));
    Head->data = A[0];
    Head->next = Head;
    last = Head;

    for (int i = 1; i < n; i++) {
        t = (struct Node *)malloc(sizeof(struct Node));
        t->data = A[i];
        t->next = Head;
        last->next = t;
        last = t;
    }
}
```

---

12. Explanation of the Creation Code

Empty Array Check

```
if (n <= 0) {
    Head = NULL;
    return;
}
```

If there are no values, there is no list to create.

So `Head` becomes `NULL`.

Create the First Node

```
Head = (struct Node *)malloc(sizeof(struct Node));
Head->data = A[0];
Head->next = Head;
last = Head;
```

This creates the first node.

If `A[0]` is `2`, then:

```
Head
 |
 v
```

```
2
^|
||
|_|
```

Important line:

```
Head->next = Head;
```

This makes the first node point to itself.

So even a one-node list is circular.

Create the Remaining Nodes

```
for (int i = 1; i < n; i++) {
    t = (struct Node *)malloc(sizeof(struct Node));
    t->data = A[i];
    t->next = Head;
    last->next = t;
    last = t;
}
```

Each new node is attached after `last`.

The new node points back to `Head`.

Then `last` moves to the new node.

The important pattern is:

```
t->next = Head;
last->next = t;
last = t;
```

Meaning:

```
Make the new node circular.
Attach it after the old last node.
Move last to the new node.
```

13. Dry Run of `create`

Array:

```
A = {2, 3, 4}
```

Step 1: Create First Node with `2`

Code:

```
Head->data = 2;
Head->next = Head;
last = Head;
```

Picture:

```
Head
 |
 v
2
^|
||
|_|
```

This is a one-node circular list.

`Head` and `last` both point to node `2`.

Step 2: Add `3`

Create a new node:

```
3
```

Code:

```
t->data = 3;
t->next = Head;
```

```
last->next = t;
last = t;
```

Picture:

```
Head
 |
 v
2 -> 3
^    |
|____|
```

Now:

```
2 points to 3.
3 points back to 2.
last points to 3.
```

Step 3: Add `4`

Code:

```
t->data = 4;
t->next = Head;
last->next = t;
last = t;
```

Picture:

```
Head
 |
 v
2 -> 3 -> 4
^         |
|_________|
```

Final list:

```
2 -> 3 -> 4 -> back to 2
```

14. Displaying a Circular Singly Linked List

Displaying means printing all node values.

For this circular list:

```
Head
 |
 v
10 -> 20 -> 30
^           |
|___________|
```

The output should be:

```
10 20 30
```

It should not print forever.

---

15. Why Normal Display Does Not Work

For a normal singly linked list, display uses:

```c
while (p != NULL) {
    printf("%d ", p->data);
    p = p->next;
}
```

This works for a linear list because the last node points to `NULL`.

But in a circular singly linked list, the last node points back to `Head`.

So `p` may never become `NULL`.

That means this code can run forever:

```c
while (p != NULL) {
    printf("%d ", p->data);
```

```
    p = p->next;
}
```

This is one of the most important warnings in Module 15.

---

16. Correct Display Rule

For a circular singly linked list, the display should stop when the pointer comes back to `Head`.

The logic is:

```
Start at Head.
Print the current node.
Move to the next node.
Stop when you return to Head.
```

The stopping condition is not:

```
p != NULL
```

The stopping condition is:

```
p != Head
```

But there is one important detail:

```
At the beginning, p is already equal to Head.
```

So we must print first and check later.

That is why we use a `do-while` loop.

---

17. Iterative Display Code

```c
void Display(struct Node *h) {
    struct Node *p = h;

    if (h == NULL) {
```

```
        return;
    }

    do {
        printf("%d ", p->data);
        p = p->next;
    } while (p != h);
}
```

---

18. Explanation of Iterative Display Code

Function Header

```
void Display(struct Node *h)
```

The function receives a pointer to the starting node.

Usually, we call it like this:

```
Display(Head);
```

Temporary Pointer

```
struct Node *p = h;
```

`p` is used to move through the circular list.

We do not move `Head` directly.

Empty List Check

```
if (h == NULL) {
    return;
}
```

If the list is empty, there is nothing to print.

This prevents unsafe access like:

```
p->data
```

when `p` is `NULL`.

Do-While Loop

```c
do {
    printf("%d ", p->data);
    p = p->next;
} while (p != h);
```

This means:

```
Print the current node.
Move to the next node.
Repeat until p comes back to h.
```

---

19. Why `do-while` Is Used

A `while` loop checks the condition first.

A `do-while` loop runs the body first, then checks the condition.

This matters because at the start:

```
p == Head
```

If we wrote this:

```c
while (p != Head) {
    printf("%d ", p->data);
    p = p->next;
}
```

The loop would not run even once.

Why?

Because `p` is already equal to `Head` at the beginning.

Correct approach:

```
do {
    printf("%d ", p->data);
    p = p->next;
} while (p != Head);
```

This allows the first node to be printed before checking whether we have returned to the start.

Simple rule:

```
Circular display needs do-while because the starting pointer and stopping
pointer are the same.
```

---

20. Dry Run of Iterative Display

List:

```
Head
 |
 v
10 -> 20 -> 30
^           |
|___________|
```

Call:

```
Display(Head);
```

Initial state:

```
p points to 10
h points to 10
```

Trace:

Step	`p` points to	Action
1	10	Print 10, move to 20
2	20	Print 20, move to 30
3	30	Print 30, move to 10
4	10	`p == h`, stop

Output:

```
10 20 30
```

Important:

```
The function does not print 10 again after returning to Head.
```

---

21. Full Working C Program

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *Head = NULL;

void create(int A[], int n) {
    if (n <= 0) {
        Head = NULL;
        return;
    }

    struct Node *t;
    struct Node *last;

    Head = (struct Node *)malloc(sizeof(struct Node));
    Head->data = A[0];
    Head->next = Head;
    last = Head;
```

```
    for (int i = 1; i < n; i++) {
        t = (struct Node *)malloc(sizeof(struct Node));
        t->data = A[i];
        t->next = Head;
        last->next = t;
        last = t;
    }
}

void Display(struct Node *h) {
    struct Node *p = h;

    if (h == NULL) {
        return;
    }

    do {
        printf("%d ", p->data);
        p = p->next;
    } while (p != h);
}

int main() {
    int A[] = {2, 3, 4};

    create(A, 3);

    Display(Head);

    return 0;
}
```

Expected output:

```
2 3 4
```

---

22. Recursive Display for a Circular Singly Linked List

In a normal singly linked list, recursive display stops at:

```
p == NULL
```

Example normal recursive display:

```c
void RDisplay(struct Node *p) {
    if (p != NULL) {
        printf("%d ", p->data);
        RDisplay(p->next);
    }
}
```

This does not work directly for a circular linked list because `p` may never become `NULL`.

In a circular linked list, recursion must stop when the function reaches `Head` again.

But there is a problem.

At the first call:

```c
RDisplay(Head);
```

The pointer is already at `Head`.

If we stop whenever `h == Head`, the function will stop immediately and print nothing.

So we need a way to say:

```
First time at Head: print it.
Second time at Head: stop.
```

This is why we use a static flag.

---

23. Recursive Display Code

```c
void RDisplay(struct Node *h) {
    static int flag = 0;

    if (h != Head || flag == 0) {
        flag = 1;
        printf("%d ", h->data);
        RDisplay(h->next);
    }
```

```
    flag = 0;
}
```

---

24. Meaning of the Static Flag

The variable:

```
static int flag = 0;
```

is used to remember whether recursion has already started.

A normal local variable is recreated every time a function is called.

A static local variable keeps its value between function calls.

That is useful here because the recursive calls need to know whether this is the first visit to `Head` or not.

---

25. Recursive Display Logic in Simple Words

The condition is:

```
if (h != Head || flag == 0)
```

This means:

```
Continue if h is not Head.
Also continue if h is Head but this is the first call.
```

At the beginning:

```
h == Head
flag == 0
```

So the first node is printed.

Then:

```
flag = 1;
```

Now the function knows traversal has started.

When recursion eventually reaches `Head` again:

```
h == Head
flag == 1
```

The condition becomes false.

So recursion stops.

---

26. Dry Run of Recursive Display

List:

```
Head
 |
 v
10 -> 20 -> 30
^           |
|___________|
```

Call:

```
RDisplay(Head);
```

Initial value:

```
flag = 0
```

Trace:

Call	`h` points to	`flag` before check	Action
1	10	0	Print 10, set flag to 1, call 20
2	20	1	Print 20, call 30

Call	`h` points to	`flag` before check	Action
3	30	1	Print 30, call 10
4	10	1	Stop because we returned to Head

Output:

```
10 20 30
```

---

27. Important Note About Resetting `flag`

The code ends with:

```
flag = 0;
```

This resets the flag after the recursive display is finished.

Why is this useful?

Because if we call `RDisplay(Head)` again later, it should work again from the beginning.

Without resetting `flag`, a later call may think traversal has already started.

---

28. Complexity Analysis

Creation Complexity

Creating a circular linked list with `n` nodes takes:

```
Time: O(n)
```

Why?

Because each array value becomes one node.

It also uses:

```
Space: O(n)
```

Why?

Because `n` nodes are created in heap memory.

Iterative Display Complexity

Displaying a circular linked list takes:

```
Time: O(n)
```

Why?

Because each node is printed once.

It uses:

```
Extra space: O(1)
```

Why?

Because only one temporary pointer `p` is used.

Recursive Display Complexity

Recursive display takes:

```
Time: O(n)
```

Why?

Because each node is printed once.

It uses:

```
Extra space: O(n)
```

Why?

Because each recursive call uses stack memory.

Complexity Table

Operation	Time complexity	Extra space	Reason
Create circular list	`O(n)`	`O(n)`	Creates `n` nodes
Iterative display	`O(n)`	`O(1)`	Uses one pointer
Recursive display	`O(n)`	`O(n)`	Uses recursion stack

---

29. Common Beginner Mistakes

Mistake 1: Setting the Last Node to `NULL`

Wrong:

```
last->next = NULL;
```

This creates a normal linear list, not a circular list.

Correct:

```
last->next = Head;
```

This makes the last node point back to the first node.

Mistake 2: Using Normal Display Logic

Wrong:

```c
while (p != NULL) {
    printf("%d ", p->data);
    p = p->next;
}
```

This may run forever because `p` may never become `NULL`.

Correct:

```
do {
    printf("%d ", p->data);
    p = p->next;
} while (p != Head);
```

Mistake 3: Forgetting the Empty List Case

Wrong idea:

```
printf("%d", Head->data);
```

This is unsafe if:

```
Head == NULL
```

Correct:

```
if (Head == NULL) {
    return;
}
```

Always check whether the list is empty before accessing node data.

Mistake 4: Confusing Empty List and One-Node List

Empty list:

```
Head == NULL
```

One-node circular list:

```
Head != NULL
Head->next == Head
```

These are not the same.

Mistake 5: Using a `while` Loop Incorrectly for Display

Wrong:

```
while (p != Head) {
    printf("%d ", p->data);
    p = p->next;
}
```

Why wrong?

Because at the start:

```
p == Head
```

So the loop does not run.

Correct:

```
do {
    printf("%d ", p->data);
    p = p->next;
} while (p != Head);
```

Mistake 6: Forgetting Why the Recursive Flag Is Needed

Wrong idea:

```
if (h == Head) {
    return;
}
```

This stops immediately at the first call.

Correct idea:

```
Allow the first visit to Head.
Stop on the second visit to Head.
```

That is why the static flag is used.

---

30. Key Code Patterns to Memorize

Empty Standard Circular List

```
Head = NULL;
```

One-Node Circular List

```
Head->next = Head;
```

Attach New Node at End During Creation

```
t->next = Head;
last->next = t;
last = t;
```

Iterative Circular Display

```
do {
    printf("%d ", p->data);
    p = p->next;
} while (p != Head);
```

Recursive Circular Display Condition

```
if (h != Head || flag == 0)
```

---

31. Mental Model

Think of a normal singly linked list as a road:

```
10 -> 20 -> 30 -> NULL
```

You eventually reach the end.

Think of a circular singly linked list as a track:

```
10 -> 20 -> 30
^           |
|___________|
```

If you keep walking, you return to where you started.

So for circular lists:

```
NULL is not the stopping point.
Returning to Head is the stopping point.
```

---

32. Final Module 15 Summary

A circular singly linked list is a singly linked list where the last node points back to the first node.

Normal singly linked list:

```
10 -> 20 -> 30 -> NULL
```

Circular singly linked list:

```
10 -> 20 -> 30
^           |
|___________|
```

The most important circular link is:

```
last->next = Head;
```

In a standard circular list:

```
Head == NULL
```

means the list is empty.

A one-node circular list has:

```
Head->next = Head;
```

A multiple-node circular list has:

```
last->next = Head;
```

Display must not use:

```
p != NULL
```

Instead, circular display stops when the pointer comes back to `Head`.

Correct display pattern:

```c
do {
    printf("%d ", p->data);
    p = p->next;
} while (p != Head);
```

Recursive display needs a static flag because the function starts at `Head` and must only stop when it reaches `Head` again.

The core lesson of Module 15 is:

```
In a circular singly linked list, the list does not end at NULL.
The traversal ends when we return to Head.
```

---

33. Quick Mastery Checks

Use these questions to test your understanding.

1. What does the last node point to in a circular singly linked list?
2. Why is `p != NULL` wrong for circular display?
3. Why do we use a `do-while` loop for circular display?
4. What is the difference between `Head == NULL` and `Head->next == Head`?
5. Why does recursive circular display need a static flag?
6. What does `last->next = Head` do?
7. What is the difference between a standard circular list and a dummy-head circular list?
8. What is the time complexity of displaying a circular linked list?

34. Minimal Code You Should Be Able to Write From Memory

```c
void Display(struct Node *h) {
    struct Node *p = h;

    if (h == NULL) {
        return;
    }

    do {
        printf("%d ", p->data);
        p = p->next;
    } while (p != h);
}
```

This function captures the main idea of Module 15:

```
Start at Head.
Print each node once.
Stop when you come back to Head.
```